

MANUAL BOOK

# BUKU PANDUAN PENGGUNAAN

---

*Z-80 Simulator Core Logic*

*Media Pembelajaran Interaktif Mikroprosesor Zilog*

*Z-80 Berbasis Web & AI*

**Nama Aplikasi:** Z-80 Simulator Core Logic (Z80 Sim)

**Jenis Produk:** Media Pembelajaran Interaktif / Simulator Mikroprosesor (PWA)

**Sasaran:** Mahasiswa Pend. Teknik Elektro / Informatika

**Mata Kuliah:** Sistem Mikroprosesor

**Model Pengembangan:** R&D (Research and Development)

<https://z80-simulation.web.app>

# BUKU PANDUAN PENGGUNAAN (MANUAL BOOK)

## Z-80 SIMULATOR CORE LOGIC

*Media Pembelajaran Interaktif Mikroprosesor Zilog Z-80 Berbasis Web & AI*

**IDENTITAS SKRIPSI & PRODUK** \* Nama Aplikasi: Z-80 Simulator Core Logic (Z80 Sim) \* Jenis Produk: Media Pembelajaran Interaktif / Software Simulator Mikroprosesor (PWA) \* URL Aplikasi: <https://z80-simulation.web.app> \* Sasaran Pengguna: Mahasiswa Pendidikan Teknik Elektro / Teknik Informatika (Mata Kuliah Sistem Mikroprosesor) \* Model Pengembangan: R&D (Research and Development)

## DAFTAR ISI

1. [BAB I: PENDAHULUAN](#)
2. 1.1 Latar Belakang & Deskripsi Media
3. 1.2 Keunggulan Media Simulator
4. 1.3 Spesifikasi Sistem yang Direkomendasikan
5. [BAB II: PENGENALAN ANTARMUKA \(UI OVERVIEW\)](#)
6. 2.1 Peta Tata Letak Antarmuka
7. 2.2 Control Panel (Eksekusi & Toolbar)
8. 2.3 Code Editor (Assembly Editor)
9. 2.4 Register Dashboard & Flag Viewer
10. 2.5 Memory Viewer & Stack Viewer
11. [BAB III: PANDUAN OPERASIONAL BERTAHAP \(STEP-BY-STEP\)](#)
12. 3.1 Menulis & Membuka Kode Assembly
13. 3.2 Melakukan Assemble & Load Program ( `Ctrl + L` )
14. 3.3 Eksekusi Langkah-demi-Langkah / Step ( `Ctrl + S` )
15. 3.4 Eksekusi Otomatis / Run ( `Ctrl + R` ) & Pengaturan Kecepatan

- 16. 3.5 Reset CPU (  )
  - 17. 3.6 Mengimpor & Mengekspor File (.asm)
  - 18. [BAB IV: MEMANFAATKAN FITUR PENDUKUNG & AI ANALYZER](#)
  - 19. 4.1 Menggunakan Preset Contoh Program Siap Pakai
  - 20. 4.2 Menggunakan Panel Materi Dasar Z-80 & Tombol "Coba"
  - 21. 4.3 Menggunakan Engine Linter (Analisis Statis)
  - 22. 4.4 Menggunakan AI Deep Scan (Google Gemini Mentor)
  - 23. [BAB V: MODUL PRAKTIKUM CONTOH KASUS](#)
  - 24. 5.1 Praktikum 1: Penjumlahan Aritmatika & Observasi Register A
  - 25. 5.2 Praktikum 2: Deteksi Boundary Overflow & Carry Flag
  - 26. 5.3 Praktikum 3: Operasi Pengulangan (Looping) & Zero Flag
  - 27. [BAB VI: TROUBLESHOOTING & PETUNJUK ERROR](#)
  - 28. 6.1 Parse Error (Kesalahan Sintaks)
  - 29. 6.2 Infinite Loop (Loop Tanpa Henti)
  - 30. 6.3 Halted State & Reset
- 

## BAB I: PENDAHULUAN

---

### 1.1 Latar Belakang & Deskripsi Media

Mikroprosesor Zilog Z-80 merupakan arsitektur 8-bit klasik yang menjadi standar pedagogis fundamental dalam mempelajari kerja internal CPU, register, eksekusi instruksi, dan arsitektur memori pada mata kuliah Sistem Mikroprosesor.

**Z-80 Simulator Core Logic** hadir sebagai perangkat lunak edukatif berbasis web yang mengemulasikan mikroprosesor Z-80 secara visual, interaktif, dan real-time. Media ini dirancang khusus untuk mendukung pembelajaran mandiri (*self-paced learning*) tanpa mengharuskan mahasiswa menginstal *software* atau *toolchain* yang rumit.

### 1.2 Keunggulan Media Simulator

-  **Tanpa Instalasi (Cross-Platform):** Dijalankan langsung melalui web browser di laptop, komputer desktop, maupun smartphone/tablet.
-  **Progressive Web App (PWA):** Dapat diinstal ke layar utama perangkat dan digunakan dalam kondisi **Offline**.
-  **Visualisasi Real-Time:** Perubahan nilai Register (A, B, C, D, E, H, L, F, SP, PC) dan Memori divisualisasikan dengan animasi warna secara langsung.

-  **AI Adaptive Feedback Dua Tahap:**
- *Engine Linter*: Memberikan umpan balik instan dan skor kesehatan kode (0–100).
- *AI Deep Scan (Gemini)*: Memberikan penjelasan pedagogis kontekstual jika mahasiswa mengalami kesulitan.
-  **Integrasi Panel Materi**: Materi dasar instruksi Z-80 lengkap dengan contoh yang dapat langsung dicoba ke editor dengan 1 klik.

### 1.3 Spesifikasi Sistem yang Direkomendasikan

- **Perangkat**: PC / Laptop / Tablet / Smartphone.
- **Peramban Web**: Google Chrome, Mozilla Firefox, Microsoft Edge, atau Safari versi terbaru.
- **Resolusi Layar**: Minimal 1280×720 (PC/Laptop) atau Layar Smartphone standar.
- **Koneksi Internet**: Diperlukan saat pertama membuka web & saat menggunakan AI Deep Scan (setelah terinstal PWA, emulator dasar dapat berjalan offline).

## BAB II: PENGENALAN ANTARMUKA (UI OVERVIEW)

Antarmuka Z-80 Simulator dibagi menjadi 2 area utama: **Area Kerja Kiri** (Editor & Panel Pendukung) dan **Area Informasi Kanan** (Register & Memori).



### 2.1 Control Panel (Eksekusi & Toolbar)

Terletak di bagian bawah layar (pada mode PC) untuk mengontrol jalannya simulasi: \* ⚡ **Load** (**ctrl+L**): Mengompilasi (*assemble*) kode di editor ke dalam memori simulasi CPU. \* ▶▶ **Step** (**ctrl+S**): Mengeksekusi **satu baris instruksi** saja untuk mengamati pergerakan register per baris. \* ▶

**Run ( Ctrl+R )**: Mengeksekusi seluruh program dari awal hingga selesai / bertemu instruksi **HALT**. \*   
**Reset**: Mengembalikan seluruh isi register, flag, dan PC ke kondisi awal ( 0000H ). \*  **Export /**   
**Import**: Menyimpan kode ke file **.asm** lokal atau mengunggah file **.asm** dari komputer. \*  **Contoh**: Menampilkan *dropdown* pilihan 7 preset program contoh siap pakai. \*  **Speed Slider**: Mengatur kecepatan simulasi saat tombol **Run** diaktifkan (Slow → Normal → Fast → Turbo).

## 2.2 Code Editor (Assembly Editor)

- Dilengkapi dengan **Syntax Highlighting** (pewarnaan kode otomatis untuk Mnemonic, Operand, Komentar **;**, dan Label).
- Penomoran baris (*Line Numbers*) dan penanda baris instruksi yang sedang dieksekusi (*Program Counter Highlight*).
- Mendukung fitur *Undo / Redo* serta pemuatan beberapa tab file.

## 2.3 Register Dashboard & Flag Viewer

Menampilkan status terkini dari seluruh register CPU Z-80: \* **Main Registers**: **A** (Accumulator), **B**, **C**, **D**, **E**, **H**, **L**. \* **Alternate Registers**: **A'**, **B'**, **C'**, **D'**, **E'**, **H'**, **L'**. \* **Special Registers**: **PC** (Program Counter) dan **SP** (Stack Pointer). \* **Flag Register (F)**: \* **SF** (Sign Flag) — Bernilai **1** jika hasil ber-bit 7 positif/negatif. \* **ZF** (Zero Flag) — Bernilai **1** jika hasil operasi bernilai **0**. \* **CF** (Carry Flag) — Bernilai **1** jika terjadi limpahan (*overflow/underflow*). \* **Toggle Format Tampilan**: Nilai register dapat diubah format tampilannya antara **HEX** (Hexadesimal), **DEC** (Desimal), atau **BIN** (Biner). \* **Animasi Flash**: Register yang baru saja berubah nilainya akan menyala dengan **sorotan warna kuning** selama 0,6 detik.

## 2.4 Memory Viewer & Stack Viewer

- Memory Viewer**: Menampilkan tabel Hex Dump memori 64KB (alamat **0000H** s.d. **FFFFH**) beserta representasi karakter ASCII di sisi kanan.
- PC & SP Highlight**: Baris memori yang ditunjuk oleh **PC** di-highlight warna **Biru**, sedangkan yang ditunjuk **SP** (Stack Pointer) di-highlight warna **Ungu**.

# BAB III: PANDUAN OPERASIONAL BERTAHAP (STEP-BY-STEP)

Berikut adalah panduan praktis dari awal membuka aplikasi hingga mengeksekusi program.

## 3.1 Menulis & Membuka Kode Assembly

- Buka peramban web dan akses <https://z80-simulation.web.app>.
- Di area **Editor Kode** (sisi kiri), ketikkan kode assembly Z-80 sederhana berikut:

```
ORG 0000H
```

```
LD A, 05H      ; Load nilai 5 (hex 05H) ke Accumulator (A)
LD B, 03H      ; Load nilai 3 (hex 03H) ke Register B
ADD A, B       ; Tambahkan B ke A (A = A + B)
HALT           ; Hentikan CPU
```

**Tips Sintaks:** \* Nilai Hexadesimal disarankan diakhiri huruf **H** (misal: **05H**, **FFH**). Jika angka hex diawali huruf (A–F), tambahkan angka **0** di depannya (misal: **0FFH**). \* Gunakan titik koma **;** untuk memberikan komentar/penjelasan baris.

### 3.2 Melakukan Assemble & Load Program ( **Ctrl + L** )

1. Setelah kode selesai diketik, klik tombol ⚡ **Load** pada Control Panel bagian bawah (atau tekan tombol kombinasi **Ctrl + L**).
2. Perhatikan panel **Execution Log** di bagian bawah editor. Jika berhasil, akan muncul pesan berwarna hijau: ✓ Program loaded successfully (4 instructions)
3. Status CPU di bagian kanan atas akan berubah menjadi **"Ready"** dan **PC** berada pada alamat **0000H**.

### 3.3 Eksekusi Langkah-demi-Langkah / Step ( **Ctrl + S** )

Mode ini sangat direkomendasikan untuk memahami pergerakan register secara observasional:

1. Klik tombol ➡ **Step** (atau tekan **Ctrl + S**).
2. **Pengamatan Step 1 ( LD A, 05H ):**
3. Lihat **Register Dashboard**: Register **A** berubah nilainya menjadi **05** (disertai animasi flash kuning).
4. Nilai **PC** berpindah dari **0000H** ke **0001H**.
5. Klik tombol ➡ **Step** kedua kalinya ( LD B, 03H ):
6. Register **B** berubah nilainya menjadi **03**.
7. Nilai **PC** berpindah ke **0002H**.
8. Klik tombol ➡ **Step** ketiga kalinya ( ADD A, B ):
9. Register **A** berubah nilainya dari **05** menjadi **08** ( $5 + 3 = 8$ ).
10. Perhatikan Flag **ZF** tetap **0**, **CF** tetap **0**.
11. Klik tombol ➡ **Step** keempat kalinya ( HALT ):
12. CPU memasuki kondisi **Halted** (Eksekusi Selesai).

### 3.4 Eksekusi Otomatis / Run ( **Ctrl + R** ) & Pengaturan Kecepatan

1. Klik tombol  **Reset** terlebih dahulu untuk mengembalikan kondisi awal.
2. Geser **Speed Slider** ke posisi **Normal** atau **Slow**.
3. Klik tombol  **Run** (atau tekan **Ctrl + R**).
4. Simulator akan mengeksekusi baris demi baris secara otomatis berurutan hingga bertemu perintah **HALT**.

### 3.5 Reset CPU ( **Reset** )

Jika ingin mengulang simulasi dari awal: \* Klik tombol  **Reset** (berwarna merah di pojok kanan bawah).

\* Nilai semua register **A, B, C, D, E, H, L**, Flag, dan **PC** akan kembali bersih ke **00H** / **0000H**.

### 3.6 Mengimpor & Mengekspor File (.asm)

- **Menyimpan Kode (Export):** Klik tombol  **Export**, file bernama **program.asm** akan otomatis terunduh ke komputer Anda.
- **Membuka File Kode (Import):** Klik tombol  **Import**, pilih file ber-ekstensi **.asm** atau **.txt** dari perangkat Anda. Kode akan otomatis tampil di editor.

## BAB IV: MEMANFAATKAN FITUR PENDUKUNG & AI ANALYZER

### 4.1 Menggunakan Preset Contoh Program Siap Pakai

Jika Anda ingin mempelajari program yang lebih kompleks tanpa mengetik dari awal: 1. Klik tombol  **Contoh** pada Control Panel bawah. 2. Pilih salah satu dari 7 program yang tersedia: \* **Penjumlahan Dasar** (● Mudah) \* **Boundary Overflow/Underflow** (● Sedang) \* **Countdown Loop** (● Sedang) \* **Memory Copy** (● Sedang) \* **Fibonacci + Overflow Trap** (● Lanjut) \* **Penjumlahan Loop 1..10** (● Sedang) \* **Pencarian Nilai Maksimum Array** (● Lanjut) 3. Kode program beserta komentar lengkapnya akan otomatis termuat ke dalam editor.

### 4.2 Menggunakan Panel Materi Dasar Z-80 & Tombol "Coba"

Aplikasi menyediakan buku referensi mini di dalam UI: 1. Klik ikon  **Materi** di barisan tombol kanan atas (atau tombol hamburger di mobile). 2. Panel **Materi Dasar Z-80** akan terbuka. 3. Anda dapat mencari instruksi melalui **Kolom Pencarian** (misal: ketik **ADD** atau **PUSH**). 4. Klik pada instruksi untuk membuka detail: \* **Sintaks:** Variasi format perintah. \* **Deskripsi:** Penjelasan fungsi perintah dalam Bahasa Indonesia. \* **Efek Flag:** Penjelasan dampak terhadap Carry ( **C** ), Zero ( **Z** ), atau Sign ( **S** ). \* **Contoh:** Blok contoh kode assembly. 5. **Tombol  Coba:** Klik tombol **Coba** di samping contoh kode, maka kode contoh tersebut akan **otomatis tersalin langsung ke editor Anda!**

### 4.3 Menggunakan Engine Linter (Analisis Statis)

1. Klik ikon  **AI Analyzer** di kanan atas.
2. Engine Linter akan menganalisis struktur kode Anda secara statis.
3. **Indikator Health Score (0–100)**: Menampilkan tingkat kesehatan & efisiensi kode.
4. **Detail Peringatan**: Linter akan mendeteksi masalah umum seperti:
5. Program lupa tidak diakhiri instruksi `HALT`.
6. Adanya potensi pengulangan tanpa henti (*Infinite Loop*).
7. Instruksi yang tidak pernah dieksekusi (*Unreachable / Dead Code*).

### 4.4 Menggunakan AI Deep Scan (Google Gemini Mentor)

1. Pada Panel AI Analyzer, klik tombol  **Deep Scan (AI)**.
2. Sistem akan mengirimkan kode assembly Anda ke layanan AI Google Gemini API.
3. AI akan bertindak sebagai **Tutor / Dosen Pendamping** yang memberikan:
4. Penjelasan maksud/tujuan dari program Anda.
5. Analisis alur logika eksekusi.
6. Saran perbaikan efisiensi atau koreksi kesalahan secara ramah dan mudah dipahami.

---

## BAB V: MODUL PRAKTIKUM CONTOH KASUS

---

### 5.1 Praktikum 1: Penjumlahan Aritmatika & Observasi Register A

**Tujuan:** Memahami pengisian nilai ke register dan operasi penjumlahan akumulator.

**Kode Praktikum:**

```
ORG 0000H
    LD A, 12H      ; A = 12H (18 desimal)
    LD B, 24H      ; B = 24H (36 desimal)
    ADD A, B       ; A = 12H + 24H = 36H (54 desimal)
    HALT
```

**Langkah Tugas:** 1. Load dan lakukan **Step** sebanyak 3 kali. 2. Catat perubahan nilai Register A pada setiap langkah! 3. *Pertanyaan:* Berapakah nilai akhir Register A dalam hex dan desimal? (*Jawaban: 36H / 54*).

---



## 5.2 Praktikum 2: Deteksi Boundary Overflow & Carry Flag

**Tujuan:** Memahami keterbatasan kapasitas register 8-bit (\$00H\$ s.d. \$FFH / 255\$) dan peran Carry Flag.

**Kode Praktikum:**

```
ORG 0000H
    LD A, 0FFH      ; A = 255 (maksimal 8-bit)
    ADD A, 01H      ; A = 255 + 1 = 256 (Jebol / Overflow!)
    HALT
```

**Langkah Tugas:** 1. Load dan jalankan dengan **Step**. 2. Amati Register A pada saat instruksi `ADD A, 01H` dieksekusi. 3. *Pengamatan:* \* Nilai Register A menjadi `00H` (karena 256 meluap kembali ke 0). \* Flag **CF (Carry Flag)** berubah nilainya dari `0` menjadi `1` (Menandakan terjadi overflow!).

## 5.3 Praktikum 3: Operasi Pengulangan (Looping) & Zero Flag

**Tujuan:** Memahami mekanisme pengulangan bersyarat menggunakan `DEC` dan `JP NZ`.

**Kode Praktikum:**

```
ORG 0000H
    LD B, 03H      ; Set hitung mundur = 3
LOOP:
    DEC B          ; Kurangi B sebanyak 1
    JP NZ, LOOP    ; Jika B belum 0 (Zero Flag = 0), lompat ke LOOP
    HALT
```

**Langkah Tugas:** 1. Load dan amati eksekusi **Step** pada perulangan tersebut. 2. Berapa kali perulangan terjadi sebelum CPU berpindah ke perintah `HALT`? 3. Amati perubahan Flag **ZF (Zero Flag)** saat `B` mencapai nilai `00H`!

# BAB VI: TROUBLESHOOTING & PETUNJUK ERROR

## 6.1 Parse Error (Kesalahan Sintaks)

- **Gejala:** Saat menekan tombol **Load**, muncul pesan merah `Parse error: Invalid instruction or operands`.
- **Penyebab:**

- Terdapat salah ketik instruksi (misal: `LOD A, 05H` bukannya `LD A, 05H`).
- Format nilai Hexadesimal salah (misal: menulis `FFH` tanpa angka `0` di depan → ubah menjadi `0FFH`).
- Lupa memberikan koma `,` pemisah antar operand (misal: `LD A 05H`).
- **Solusi:** Periksa kembali baris kode yang ditunjukkan pada pesan error dan perbaiki ejaannya.

## 6.2 Infinite Loop (Loop Tanpa Henti)

- **Gejala:** Saat menekan **Run**, aplikasi terus berjalan tanpa pernah berhenti ( `Executing program...` ).
- **Penyebab:** Kondisi lompatan `JP` atau `JR` tidak pernah terpenuhi untuk keluar dari perulangan, atau lupa menyertakan perintah `HALT`.
- **Solusi:** Klik tombol  **Reset**, lalu tambahkan perintah `HALT` di akhir program atau periksa syarat flag pada perulangan Anda.

## 6.3 Halted State & Reset

- **Gejala:** Tombol **Step** atau **Run** berwarna abu-abu (tidak bisa diklik).
- **Penyebab:** CPU sudah mencapai perintah `HALT` (kondisi *Halted*).
- **Solusi:** Klik tombol  **Reset** untuk mengembalikan CPU ke kondisi siap jalan.

---

*Manual Book ini disusun sebagai bagian dari Dokumentasi & Panduan Penggunaan Skripsi Z-80 Simulator Core Logic.*

---

*Manual Book ini disusun sebagai bagian dari Dokumentasi & Panduan Penggunaan Skripsi  
Z-80 Simulator Core Logic — Media Pembelajaran Interaktif Mikroprosesor Z-80 Berbasis Web & AI.*